



Assignment - 1

Q (i) Give advantages of linked list over array?

Ans. Linked list are dynamic data structure, which can grow or shrink during the execution of a program.

2. There is no need to define an initial size for a linked list.

3. There is no need to ~~defined~~ specify how many nodes are required.

3.4. Therefore, linked list have efficient memory ~~utilization~~ utilization and no wastage of memory.

5. The main advantage of a linked list over an array is insertion and Deletion operations are easier and more efficient.

6. In linked list memory is not pre-allocated.

7. Linked list provide flexibility in inserting a data item at a specified position and deletion of the data item from the given position.

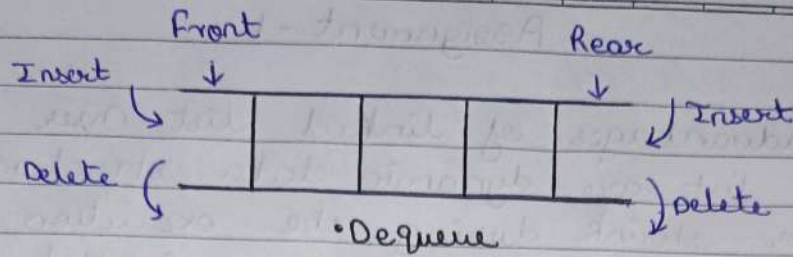
8. Many complex application can be easily carried out with linked lists such as-

- Hash Table Implementation
- Polynomial Representation
- Abstract Data Type Implementation
- S-expression
- Stack and Queue Implementation.

9. Backtracking is possible in two way linked list (Doubly linked list).

(ii) Discuss the concept and types of double ended queue?

Ans. Double Ended Queue is data structure in which the insertion and deletion operations are allowed at both ends. That means, we can insert at both front and rear position and as well we can delete from both front and rear position.



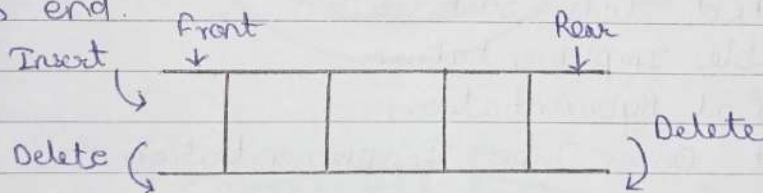
Application of Doubly Ended Queue -

1. A web browser's history. Recently visited URLs are added to the front of dequeue & the URL at the ~~end~~ back of the dequeue is removed after some specified number of insertion at the front.
2. Storing a software application's list of undo operation.
3. Money Control App.

There are two types of Double Ended Queue -

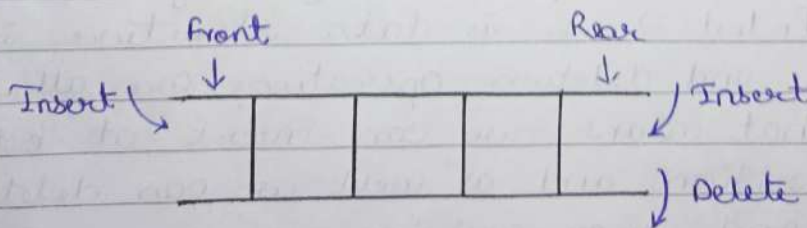
1. Input restricted double ended queue

In this type of queue, the insertion operation is allowed at only one end while deletion operation is allowed at both end.



2. Output restricted Double ended queue

In this type of queue, the deletion operation is allowed at only one end while insertion operation is allowed at both ends.





(iii)

Ans

Construct a program for Addition of 2 polynomials.

#include <stdio.h>

#include <stdlib.h>

struct Node {

int coeff;

int exp;

struct Node *next;

};

struct Node* createNode(int coeff, int exp) {

struct Node* newNode = (struct Node*)

malloc(sizeof(struct Node));

newNode->coeff = coeff;

newNode->exp = exp;

newNode->next = NULL;

return newNode;

}

void insertNode(struct Node** poly, int coeff, int exp)

{

struct Node* newNode = createNode(coeff, exp);

if (*poly == NULL) {

*poly = newNode;

} else {

struct Node* temp = *poly;

while (temp->next != NULL)

temp = temp->next;

temp->next = newNode;

}

}

struct Node* addPolynomials(struct Node* poly1,

struct Node* poly2) {

struct Node* result = NULL;

struct Node* ptr1 = poly1;

struct Node* ptr2 = poly2;

```

while (ptr1 != NULL && ptr2 != NULL) {
    if (ptr1->exp > ptr2->exp) {
        insertNode(&result, ptr1->coeff, ptr1->exp);
        ptr1 = ptr1->next;
    } else if (ptr1->exp < ptr2->exp) {
        insertNode(&result, ptr2->coeff, ptr2->exp);
        ptr2 = ptr2->next;
    } else {
        int sumCoeff = ptr1->coeff + ptr2->coeff;
        if (sumCoeff != 0)
            insertNode(&result, sumCoeff, ptr1->exp);
        ptr1 = ptr1->next;
        ptr2 = ptr2->next;
    }
}

```

```

while (ptr1 != NULL) {
    insertNode(&result, ptr1->coeff, ptr1->exp);
    ptr1 = ptr1->next;
}

while (ptr2 != NULL) {
    insertNode(&result, ptr2->coeff, ptr2->exp);
    ptr2 = ptr2->next;
}

return result;

```

```

Void displayPolynomial (Struct Node* poly) {
    if (poly == NULL) {
        printf("0");
        return;
    }
}

```



```
Struct Node* temp = poly;
while (temp != NULL) {
    if (temp -> coeff > 0 && temp != poly)
        printf (" + ");
    if (temp -> coeff < 0)
        printf (" - ");
    else if (temp != poly)
        printf (" + ");
    printf ("%d x^%d", abs(temp -> coeff),
            temp -> exp);
    temp = temp -> next;
}
printf ("\n");
}

int main () {
    Struct Node* poly1 = NULL;
    Struct Node* poly2 = NULL;
    Struct Node * Sum = NULL;

    insertNode (& poly1, 5, 2);
    insertNode (& poly1, 4, 1);
    insertNode (& poly1, 2, 0);

    insertNode (& poly2, 3, 2);
    insertNode (& poly2, 2, 1);
    insertNode (& poly2, 1, 0);

    printf ("First Polynomial : ");
    displayPolynomial (poly1);

    printf ("Second Polynomial : ");
    displayPolynomial (poly2);

    Sum = addPolynomials (poly1, poly2);
```

printf("Sum of the Polynomials: ");
displayPolynomial (sum);

return 0;

}

Output :

First Polynomial : $5x^2 + 4x^1 + 2x^0$

Second Polynomial : $3x^2 + 2x^1 + 1x^0$

Sum of the Polynomial : $8x^2 + 6x^1 + 3x^0$

(iv) Evaluate the expression $54321 * + * +$ and give the algorithm.

Scanned Element	Operation	Stack
5	Push 5	5
4	Push 4	5 4
3	Push 3	5 4 3
2	Push 2	5 4 3 2
1	Push 1	5 4 3 2 1
*	Pop 1 & 2 Push $1 * 2$	5 4 3 2
+	Pop 2 & 3 Push $2 + 3$	5 4 5
*	Pop 5 & 4 Push $5 * 4$	5 20
+	Pop 20 & 5 Push $20 + 5$	25

Final result : 25

Algorithm -

The given expression is written in postfix notation

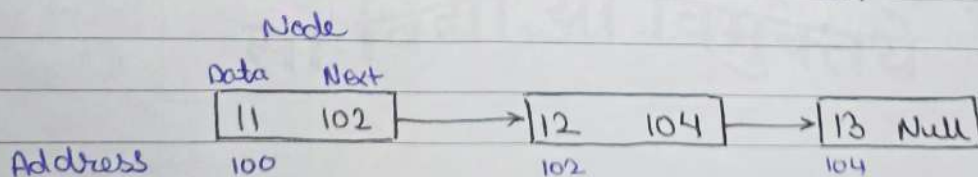
1. Initialize an empty stack.
2. Scan the expression from left to right:
 - If the character is a digit, push it onto the stack.
 - If the character is an operator (+, *, etc):
 - Pop the required number of operand from the Stack (For + & * pop two operands)
 - Perform the operation
 - Push the result back onto the stack.
3. After the entire expression has been processed, the value remaining on the Stack is the result.

v) Explain the types of linked list.

Ans There are three types of linked List -

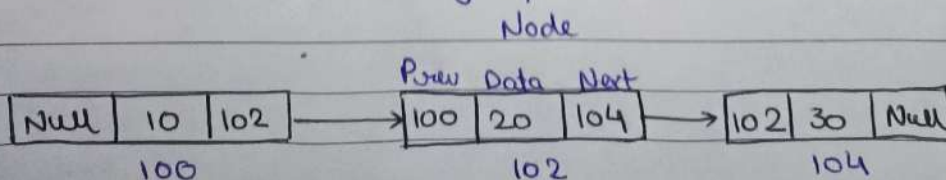
1. Singly linked List:

Singly linked list is the basic form of linked list. In this linked list, every node is made up of two parts: data and next. The next part of last node always contain null value. It is a one way traversing list, which means we can traverse only in forward direction. We cannot traverse in backward direction.



2. Doubly linked list:

Doubly linked list is the list in which every node contain three parts: data previous and next. Data contain the actual element while the previous and next parts contain the addresses of previous and next nodes.



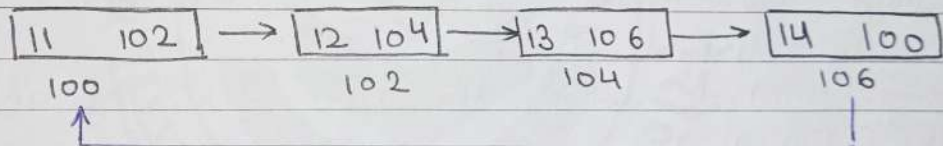
Operations-

- Inserting at end
- Inserting at beginning
- Inserting in middle
- Display
- Count
- Delete (Any node)

3. Circular Linked list:

Circular linked list is the linked list in which next part of last node contains address of first node to form a circle.

In circular linked list, we use the concept of Queue. Hence it can also be considered as Queue as linked list.



operation-

1. Inserting Node in Circular linked list
2. Deleting Node from Circular linked list
3. Displaying Data of Circular Linked list.